

Architecture Audit

Proprely — Next.js / Supabase / Vercel

Prepared: 2026-05-26 · Audience: Management & team leads · AI-assisted review

Overall Picture

The app is built on solid modern tools (Next.js, Supabase, Vercel, Sentry) and the code is generally well-structured. Most issues below are not emergencies — they are the kind of technical debt that accumulates during fast-moving early development and should be addressed before the user base grows.

Architecture & Scalability Issues

1. All Data Loads at Once on Every Page Visit

MEDIUM

Every time a user opens the app, the server fetches the entire company dataset — all missions, clients, agents, quotes, hours — in one large batch before showing anything.

Risk: Works fine today with small datasets. As companies grow, page load times will increase noticeably. There is also a Supabase query cost concern at scale.

Fix: Load only the data needed for the current page, and fetch the rest on demand. Start with the heaviest pages (planning, cockpit).

2. Backend is Built Inside the Frontend Framework

MEDIUM

All backend logic — API routes, database access, scheduled jobs, email sending, business rules — lives inside Next.js, a frontend framework. NestJS is a dedicated backend framework built for this kind of structured, scalable server-side work.

Risk: As the product grows, mixing frontend and backend creates friction: harder to scale each side independently, harder to test business logic in isolation, and harder to onboard backend-focused developers.

Fix: Plan a gradual migration of backend logic into a dedicated NestJS service. Start by extracting the most complex areas (data actions, cron jobs, email workflows) into a standalone API.

3. No Standard Format for API Requests and Responses

MEDIUM

Different parts of the app return data in completely different shapes. Server actions use at least three separate result types defined independently. API routes return raw objects — some return HTTP 200 even when reporting an error.

Risk: Every frontend component has to guess what shape it will receive. This leads to inconsistent error messages, bugs in specific flows, and makes the API impossible to document or test systematically.

Fix: Define one shared response envelope across all API routes and server actions. Enforce it via a shared wrapper utility. Low disruption, high payoff in developer clarity.

4. The Entire App is Hard-Coupled to Supabase

MEDIUM

183 direct database calls spread across 39 files — including page components, UI components, layouts, background jobs, and API routes. There is no abstraction layer between the app and the database.

Risk: No clean way to swap, mock, or extend the data layer without touching dozens of files. Migrating away from Supabase would be enormous. Automated testing requires hitting a live Supabase instance.

Fix: Introduce a repository layer — focused service files that own all database interactions. Supabase becomes a swappable detail. Best done alongside the NestJS migration (issue 2).

5. Updating Small Data Forces a Full Page Re-render

MEDIUM

Every time a user saves a change, the app re-fetches and re-renders the entire page from scratch. Changing one mission status reloads the entire cockpit view with all missions, agents, and operational items.

Risk: The screen flickers on every save. As data grows, reloads get slower. The app feels unresponsive. Unnecessary database reads on every mutation.

Fix: Update only the affected piece of data in the local store immediately after a successful save. The Zustand store is already in place — wire mutations to local updates instead of full-page revalidation.

6. Form Validation is Hand-Built When Better Tools Are Already Installed

LOW

The app uses HTML5 browser attributes plus a custom hook for validation. Meanwhile, react-hook-form and @hookform/resolvers are already installed as dependencies but never used anywhere in the codebase.

Limitation: On complex forms (e.g. pricing settings with 10+ fields), users only find out something is wrong after submitting. Errors come back as a single message, not inline under the specific field.

Recommendation: Adopt React Hook Form with Zod resolvers for complex multi-field forms. The packages are already installed. Start with pricing settings and onboarding steps.

7. Shared Components Exist but Are Not Consistently Used

LOW

The project has a shared component library, but the same patterns are recreated inline instead: empty states written 25+ ways, error boxes hand-copied 22 times, loading spinners duplicated 25 times, status badges recreated 16 times.

Risk: A design change must be applied in dozens of separate places. Visual inconsistencies across the app, and future UI updates are slow and error-prone.

Recommendation: Establish a team rule: check the shared components folder before writing new UI markup. When a pattern appears more than twice, extract it into a shared component.

8. Soft Delete is Only Applied to Some Tables

MEDIUM

Soft delete (marking records as archived instead of deleting them) is implemented for clients, sites, and agents only. Missions, opportunities, leads, quotes, time entries, and SOPs use hard delete — permanently gone.

Risk: A manager who accidentally deletes a mission or time entries has no way to recover that data. No undo, no trash, no audit trail for these deletions.

Fix: Extend soft delete to critical tables — at minimum missions and time_entries, then quotes and opportunities. The pattern is already in the codebase and straightforward to replicate.

9. No ORM — Database Types and Schema Managed by Hand

MEDIUM

All database queries use Supabase's raw query builder. The schema is managed through 30 hand-written SQL migration files. TypeScript types are written by hand with no connection to the actual database schema.

Risk: Schema and type mismatches go undetected until runtime. A new developer must read 30 raw SQL files to understand the data model. No generated schema reference or entity map.

Recommendation: Adopt Drizzle ORM — lightweight, works natively with Supabase and Postgres, schema defined in TypeScript, migrations auto-generated. Best introduced alongside NestJS (issue 2) and repository layer (issue 4).

10. Data Fetching Has No Caching — Rebuilds Everything on Every Change

MEDIUM

Every page load fetches the entire dataset from scratch. Any save reloads the whole page. Failed saves leave the UI showing incorrect data until the user manually refreshes. Navigating back re-fetches everything again.

Risk: The app feels slow and flickers on every action. As data grows, the problem gets worse. Failed saves leave the UI in a broken state with no automatic recovery.

Recommendation: Adopt TanStack Query for server data fetching. Each page loads only what it needs. Failed saves roll back automatically. Previously visited pages load instantly from cache. Zustand stays for UI state.

Security Issues

11. Dummy Mode Disables All Security in Misconfigured Environments

HIGH

When Supabase environment variables are missing, every permission check returns owner access with no verification. RBAC, Row Level Security, and all auth gates are completely bypassed.

Risk: One misconfigured production deploy exposes all tenant data to anyone.

Fix: Add a deployment gate that fails the release if Supabase variables are missing. A runtime check already exists — it needs to also run as a CI post-deploy step.

12. Password Policy is Too Weak

MEDIUM

The minimum password length is 6 characters with no complexity requirements and no check against commonly used passwords.

Risk: For a B2B platform managing staff data, hours, and financials, this is below the acceptable minimum.

Fix: Raise the minimum to at least 12 characters. Add a basic complexity rule. Supabase allows custom password policies in Auth settings.

13. No CORS Policy on API Routes

MEDIUM

None of the API routes define Access-Control-Allow-Origin headers. Any external website can make cross-origin requests to the API from a user's browser.

Risk: Combined with session cookies, a malicious page could silently call authenticated API endpoints on behalf of a logged-in user.

Fix: Add CORS middleware in next.config.ts restricting Access-Control-Allow-Origin to the app's own domain. Public endpoints (health, cron) can stay open.

14. unsafe-inline Scripts Still Allowed in Production CSP

LOW

The Content Security Policy allows inline scripts in production (unsafe-inline in script-src). This weakens the browser's built-in protection against cross-site scripting.

Risk: If an attacker manages to inject a script, the CSP provides no barrier to stop it running.

Fix: Migrate to nonce-based CSP. Each server response generates a one-time token attached to legitimate script tags. Any injected script without the nonce is blocked. Estimated: 2-3 days.

Summary Table

#	Issue	Area	Priority
1	All data loaded at once, every page	Scalability	Medium
2	Backend mixed into frontend (Next.js vs NestJS)	Scalability	Medium
3	No standard API request/response format	Stability	Medium
4	Entire app hard-coupled to Supabase	Scalability	Medium
5	Small updates trigger full page re-render	Stability	Medium
6	Custom form validation, libraries already installed	UX	Low
7	Shared components not consistently used	Stability	Low
8	Soft delete inconsistent — critical tables use hard delete	Stability	Medium
9	No ORM — schema, types, and queries disconnected	Stability	Medium
10	No data caching — full reload on every change	Performance	Medium
11	Dummy mode disables all security (misconfigured env)	Security	High
12	Password policy too weak (6 char, no complexity)	Security	Medium
13	No CORS policy on API routes	Security	Medium
14	unsafe-inline scripts in production CSP	Security	Low

For questions about any item, reach out to the engineering lead.